Vysvětlení projektu a testů

Doprovodný dokument k semestrální práci z předmětu Testování softwaru

Michal Hrouda

Obsah

1. Popis projektu . . . 2
   1.1. Účel aplikace . . . 2
   1.2. Architektura . . . 2
   1.3. Klíčová business pravidla . . . 2
2. Testovací infrastruktura . . . 3
   2.1. `SeleniumBaseTest` . . . 3
   2.2. Page Object pattern . . . 3
   2.3. CSV data providery . . . 3
3. Jednotlivé testovací třídy . . . 4
   3.1. `IceCreamConfigurationPairwiseTests` . . . 4
   3.2. `CreateOrderProcessTests` . . . 5
   3.3. `NoAccountClientOrderTests` . . . 7
   3.4. `AuthClientOrderTests` . . . 8
   3.5. `AdminPageTests` . . . 9
4. Souhrn — mapování testů na techniky . . . 11
   4.1. Sdílené vlastnosti všech tříd . . . 11

1. Popis projektu

1.1. Účel aplikace

Testovaná aplikace je **e-shop pro objednávání zmrzliny** z více kamenných zmrzlináren. Uživatel si vybere konkrétní pobočku, do košíku přidá existující produkty nebo vlastní zmrzlinu sestavenou v konfigurátoru, a odešle objednávku k osobnímu vyzvednutí. Aplikace byla vygenerována pomocí **Claude Opus** 4 podle zadání a slouží jako předmět testování — není to produkční systém.

1.2. Architektura

Vrstva	Popis
Framework	Spring Boot 3 (Java 21), Thymeleaf šablony pro HTML
Bezpečnost	Spring Security — role `ROLE_USER` a `ROLE_ADMIN`, `/admin/**` chráněno
Perzistence	**Bez databáze** — všechna data (`shops.json`, `products.json`, `users.json`, `orders.json`) jsou JSON soubory v `./data/` přes `FileStorageService`
Košík	Drží se v `HttpSession` (klíč `"cart"`), nepersistuje se
Seed dat	`DataInitializer` při prvním spuštění vytvoří zmrzlinárny, produkty a dva testovací účty (admin / admin123, jan / jan123)
UI	Bootstrap 5 + vanilla JavaScript pro interaktivní konfigurátor a kvíz

1.3. Klíčová business pravidla

- **Jedna zmrzlinárna v košíku** — `CartService` při přidání produktu z jiné zmrzlinárny vyhazuje `IllegalStateException` se zprávou „Košík obsahuje produkty z jiné zmrzlinárny.„
- **Minimální hodnota objednávky 50 Kč** — `OrderService.createOrder()` vyhazuje `IllegalStateException` se zprávou „Minimální hodnota objednávky je 50 Kč.„ (konfigurovatelné přes `app.order.min-total`).
- **Vzorec konfigurátoru**: `kornout 15 Kč / kelímek 10 Kč + kopečky × 25 + posypky × 10 + sušenky × 15`.
- **Stavy objednávky**: `NEW → CONFIRMED → READY → PICKED_UP`, případně `CANCELLED`.
- **Známé defekty** v konfigurátoru (záměrně nedořešené pro účely testování): chybí validace maximálního počtu kopečků / posypek / sušenek a validace výčtu povolených hodnot.

2. Testovací infrastruktura

2.1. `SeleniumBaseTest`

Abstraktní základ všech Selenium testů. Pro každý test (`@BeforeEach`) spustí ChromeDriver v headless módu, po testu (`@AfterEach`) jej uzavře. Běží jako `@SpringBootTest(webEnvironment = RANDOM_PORT)` — Spring Boot nastartuje celou aplikaci na náhodném portu a ten předá testům přes `baseUrl()`.

Pomocné metody:
- `loginAs(username, password)` — obecný login,
- `loginAsAdmin()` ⟶ admin / heslo123,
- `loginAsUser()` ⟶ jan / jan123.

2.2. Page Object pattern

Každá testovaná stránka má vlastní třídu, která zapouzdřuje lokátory a interakce. Testy pak volají sémantické metody (`shopPage.addProductByName(name, qty)`) místo přímého `driver.findElement(...)`. Výhoda: když se změní HTML, opravuje se pouze Page Object, ne všechny testy.

Třída	URL	Klíčové metody
`HomePage`	`/`	`takeQuiz(s1, s2, s3)` — projde kvíz a vrátí výsledný titul
`LoginPage`	`/login`	`loginAs(u, p)`
`ShopDetailPage`	`/shop/{id}`	`addProductByName(name, qty)`, `hasAddToCartButton`, `hasCrossShopLockIcon`
`CartPage`	`/cart`	`getCartItemCount`, `clearCart`, `proceedToCheckout`
`CheckoutPage`	`/order/checkout`	`fillContactDetails`, `submit`
`OrderConfirmationPage`	`/order/confirmation/{id}`	`isOnConfirmationPage`
`MyOrdersPage`	`/order/moje`	`hasOrders`, `cancelFirstOrder`
`ConfiguratorPage`	`/configurator`	`selectCone/Cup`, `addScoopAndWait`, `selectTopping/Cookie`, `submit`

2.3. CSV data providery

Dvě třídy testů používají `@CsvFileSource` pro parametrizaci:
- `src/test/resources/pairwise/ice-cream-pairwise.csv` — 9 testovacích případů pro pairwise test konfigurátoru,
- `src/test/resources/quiz/quiz-combinations.csv` — 6 kombinací odpovědí kvízu.

3. Jednotlivé testovací třídy

3.1. `IceCreamConfigurationPairwiseTests`

3.1.1. Co testuje

Správnost výpočtu ceny v konfigurátoru vlastní zmrzliny pro pokrytí **pairwise (all-pairs)** — každá dvojice hodnot libovolných dvou vstupních faktorů je otestována alespoň jedním testem.

Faktory a hodnoty:

- obal: `kornout`, `kelimek` (2 hodnoty),
- počet kopečků: 1, 3, 5 (3 hodnoty — min, střed, max),
- počet posypek: 0, 3, 6 (3 hodnoty),
- počet sušenek: 0, 2, 5 (3 hodnoty).

Místo úplného kartézského součinu (2 × 3 × 3 × 3 = 54 kombinací) stačí 9 testovacích případů.

3.1.2. Jak je napsán

Parametrizovaný test s `@CsvFileSource` načítajícím CSV s devíti řádky (PW1..PW9). `@BeforeEach` provede `loginAsAdmin()` a otevře stránku `/configurator`. Jedna testovací metoda `pairwise_configuratorBuildsExpectedPrice` se spouští devětkrát — pro každý řádek CSV.

3.1.3. Průběh jednoho testovacího případu (stejný pro všech 9 řádků)

1. `loginAsAdmin()` — přihlásí se jako admin (login → POST `/login` → redirect `/`),
2. `configuratorPage.open(baseUrl())` — naviguje na `/configurator`,
3. podle sloupce `container` klikne buď `selectCone()` (kornout) nebo `selectCup()` (kelímek),
4. `selectScoops(scoops)` — cyklus `(scoops - 1)` × `addScoopAndWait()` (start má 1 kopeček, takže N – 1 kliknutí; metoda čeká na DOM update po každém přidání),
5. `selectFirstN(toppingCount, TOPPINGS, selectTopping)` — zaškrtne prvních N posypek ze seznamu (`Čokoládový posyp`, `Barevný posyp`, …),
6. `selectFirstN(cookieCount, COOKIES, selectCookie)` — totéž pro sušenky (`Oplatka`, `Čokoládová tyčinka`, …),
7. `configuratorPage.getTotalPriceText()` — **před odesláním** assertuje, že UI zobrazuje očekávanou cenu z CSV ve formátu `"{N},00 Kč"`,
8. `configuratorPage.submit()` — odešle formulář (POST `/configurator/add`),
9. `WebDriverWait` čeká, až se objeví buď `alert-success` nebo `alert-danger`,
10. assertuje `isSuccessDisplayed() == true` a `isErrorDisplayed() == false`.

3.1.4. Konkrétní případy z CSV

ID	Obal	Kopečky	Posypky	Sušenky	Cena
PW1	kornout	1	0	0	40 Kč
PW2	kornout	3	3	2	150 Kč
PW3	kornout	5	6	5	275 Kč
PW4	kelímek	1	3	5	140 Kč
PW5	kelímek	3	6	0	145 Kč
PW6	kelímek	5	0	2	165 Kč
PW7	kornout	1	6	2	130 Kč
PW8	kelímek	3	0	5	160 Kč
PW9	kornout	5	3	0	170 Kč

Ověření vzorce pro PW2: `15 (kornout) + 3×25 (kopečky) + 3×10 (posypky) + 2×15 (sušenky) = 15 + 75 + 30 + 30 = 150 Kč.` ✓

> **Poznámka k omezení**: faktor „které konkrétní posypky/sušenky„ není do pairwise zahrnut — test vybírá vždy prvních N položek ze seznamu. Pokrývá se tedy **kardinalita**, ne konkrétní výběr z výčtu.

3.2. `CreateOrderProcessTests`

3.2.1. Co testuje

Proces vytvoření objednávky pomocí techniky **CPT s hloubkou pokrytí** 2 (viz přednáška 3 — průchody programem). Osm testovacích případů P1..P8 odpovídá osmi cestám získaným z CPT-managera pro daný activity diagram:

ID	Scénář	Očekávaný výsledek
P1	Stejný produkt 2× (sloučení v košíku) → checkout	`/order/confirmation`
P2	Dva různé produkty ze stejné zmrzlinárny → checkout	`/order/confirmation`
P3	Pokus o přidání produktu s neexistujícím ID	Redirect `/` + „Produkt nenalezen„
P4	Množství > skladové zásoby	Redirect `/shop/{id}` + „Nedostatek zásob„
P5	Produkt z jiné zmrzlinárny než v košíku	Redirect `/shop/{id}` + „jiné zmrzlinárny„
P6	Sloučení → vyprázdnit košík → checkout	Redirect `/cart` + „Košík je prázdný„
P7	Nový produkt → vyprázdnit košík → checkout	Redirect `/cart` + „Košík je prázdný„
P8	Celková cena < 50 Kč	Redirect `/cart` + „Minimální hodnota„

3.2.2. Jak je napsán

Selenium E2E testy přihlášené jako `jan`. Samotný testovaný flow (přidání do košíku, checkout, potvrzení) probíhá **výhradně přes UI** pomocí Page Objects. `ProductService` je autowired pouze pro **přípravu testovacích dat** — pomocné metody `highStockProduct()`, `availableProducts()`, `cheapestAvailableProduct()` čtou produkty přes backend, aby test nemusel hardcodovat konkrétní ID.

Některé negativní scénáře (P3, P4, P5) vyžadují obejít UI validaci — používají pomocnou metodu `postAddToCart(productId, quantity)`, která přes `JavascriptExecutor` vytvoří a odešle hidden `<form>` s CSRF tokenem zkopírovaným z aktuální stránky. Simuluje se tím útok na endpoint `POST /cart/add/{id}` s hodnotami, které by UI nepustilo.

3.2.3. Průběh jednotlivých testů

P1 — sloučení existující položky → potvrzení

1. `@BeforeEach`: `loginAsUser()` (jan/jan123).
2. `highStockProduct(SHOP_A)` — přes `productService.getAll()` vybere produkt s největším skladem.
3. Otevře `/shop/shop-1`, `addProductByName(name, 1)`.
4. Znovu otevře `/shop/shop-1`, `addProductByName(name, 1)` — košík **slučuje** položky (qty: 1 → 2).
5. `submitValidCheckout()`: GET `/order/checkout`, vyplní `firstName/lastName/email/phone`, klik submit, čeká na URL `/order/confirmation`.
6. Assert: `isOnConfirmationPage() == true`.

P2 — přidání nové položky → potvrzení

1. Vezme první dva dostupné produkty ze SHOP_A.
2. Otevře `/shop/shop-1`, přidá produkt A (qty 1).
3. Znovu otevře `/shop/shop-1`, přidá produkt B (qty 1) — v košíku **dva** různé řádky.
4. `submitValidCheckout()`.
5. Assert: `isOnConfirmationPage() == true`.

P3 — neexistující produkt → „Produkt nenalezen„

1. Otevře home `/`.
2. `postAddToCart("this-id-does-not-exist", 1)` — JavaScriptExecutor vytvoří skrytý form s CSRF tokenem a odešle POST na `/cart/add/this-id-does-not-exist`.
3. Čeká na redirect zpět na home (URL regex `.*:\d+/?$`).
4. Assert: `.alert-danger` obsahuje `"Produkt nenalezen"`.

P4 — množství nad stav skladu → „Nedostatek zásob„

1. `highStockProduct(SHOP_A)` + `overStock = stockQuantity + 1`.
2. Otevře `/shop/shop-1`.
3. `postAddToCart(productId, overStock)` — JS form POST obejde UI validaci množství.
4. Čeká na URL obsahující `/shop/shop-1`.
5. Assert: `.alert-danger` obsahuje `"Nedostatek zásob"`.

P5 — produkt z jiné zmrzlinárny → „jiné zmrzlinárny„

1. Vezme první produkt ze SHOP_A a SHOP_B.
2. Otevře `/shop/shop-1`, `addProductByName(productA, 1)`.
3. Otevře `/shop/shop-2` (kde tlačítko „přidat do košíku„ **je skryté**, místo něj je zámek).
4. `postAddToCart(productB.id, 1)` — obejde skryté tlačítko a odešle POST ručně.
5. Čeká na URL obsahující `/shop/shop-2`.
6. Assert: `.alert-danger` obsahuje `"jiné zmrzlinárny"`.

P6 — sloučit, vyprázdnit, checkout → „Košík je prázdný„

1. `highStockProduct(SHOP_A)`, přidá 2× stejný produkt (sloučení qty: 1 → 2).
2. Otevře `/cart`, `clearCart()` (klik na tlačítko „Vyprázdnit košík„).
3. Naviguje na `/order/checkout`.
4. Backend vidí prázdný košík → redirect na `/cart`.
5. Assert: `.alert-danger` obsahuje `"Košík je prázdný"`.

P7 — přidat nový, vyprázdnit, checkout → „Košík je prázdný„

1. `highStockProduct(SHOP_A)`, přidá 1× (bez sloučení).
2. Otevře `/cart`, `clearCart()`.
3. Naviguje na `/order/checkout`.
4. Assert: redirect na `/cart`, `.alert-danger` obsahuje `"Košík je prázdný"`.

P8 — celková cena pod 50 Kč → „Minimální hodnota„

1. `cheapestAvailableProduct(SHOP_A)` — produkt s nejnižší cenou.
2. Precondition assert: `price < 50`.
3. Otevře `/shop/shop-1`, `addProductByName(name, 1)`.
4. Naviguje na `/order/checkout`, `fillValidCustomerDetails()`, `submit()`.
5. Backend (`OrderService.createOrder`) vyhodí `IllegalStateException` → redirect na `/cart`.
6. Assert: `.alert-danger` obsahuje `"Minimální hodnota"`.

3.3. `NoAccountClientOrderTests`

3.3.1. Co testuje

Funkcionality dostupné **nepřihlášenému (hostovi)** — krom konfigurátoru, který je pokryt samostatně.

1. **Kvíz pro výběr zmrzliny** — 6 kombinací odpovědí (3 × 3 kroky, ale datově orientovaně jen 6 zajímavých případů z CSV).
2. **Přidání produktu do košíku jako host** → očekává `alert-success`.
3. **Produkt z jiné zmrzlinárny** — ověření, že tlačítko „přidat do košíku„ **zmizelo** a místo něj je ikona zámku.
4. **Více kusů jednoho produktu** — košík zobrazí 1 **řádek** (nikoli 3 samostatné).
5. **Objednávka pod 50 Kč** → chyba „Minimální hodnota„.
6. **Dokončení objednávky jako host** → potvrzovací stránka.

3.3.2. Jak je napsán

Parametrizovaný test kvízu používá `@CsvFileSource` (`quiz-combinations.csv`), ostatní testy jsou obyčejné `@Test` metody. Host nikdy neloginuje — `@BeforeEach` pouze inicializuje Page Objects. Pro výběr produktu pro test s limitem ceny se v `productAndQtyForMinTotal()` spočítá minimální množství, aby se dostalo nad 50 Kč, a zkontroluje se, že produkt má dostatek na skladě.

3.3.3. Průběh jednotlivých testů

Test 1 — kvíz: správný výsledek pro kombinaci odpovědí (`@Order` 1, parametrizovaný, 6 průchodů)

Vstupy (z CSV):

step1	step2	step3	expectedTitle
sweet	sweet	sweet	Milovník/milovnice sladkého!
fruity	fruity	fruity	Ovocný/á svěžák/svěžačka!
creamy	creamy	creamy	Klasický/á krémový/á typ!
exotic	exotic	exotic	Dobrodružná duše!
sweet	sweet	fruity	Milovník/milovnice sladkého!
creamy	fruity	fruity	Ovocný/á svěžák/svěžačka!

Pro každý řádek:

1. `homePage.open(baseUrl())` — GET `/`.
2. `homePage.takeQuiz(step1, step2, step3)` — postupně kliká na elementy podle selektoru `[data-step='1'][data-value='{step1}']` atd.; po 3. kroku čte text modálu s výsledkem.
3. Assert: vrácený titul odpovídá `expectedTitle` z CSV.

Test 2 — přidání produktu do košíku jako host → úspěch (`@Order` 2)

1. `highStockProduct(SHOP_A)` — nejvíc skladový produkt v SHOP_A.
2. Otevře `/shop/shop-1`.
3. `addProductByName(name, 1)` — najde kartu produktu a klikne „Přidat do košíku„.
4. Page Object čeká na `.alert-success`.
5. Assert: `isSuccessDisplayed() == true`.

Test 3 — produkt z jiné zmrzlinárny má skryté tlačítko + zámek (`@Order` 3)

1. Vezme první produkt ze SHOP_A (`productA`) a ze SHOP_B (`productB`).
2. Otevře `/shop/shop-1`, `addProductByName(productA, 1)` — košík je vázaný na shop-1.

3. Otevře `/shop/shop-2`.
4. Assert: `hasAddToCartButton(productB.name) == false` — tlačítko není v DOM/skryté.
5. Assert: `hasCrossShopLockIcon(productB.name) == true` — místo tlačítka je ikona zámku.

Test 4 — více kusů = jeden řádek v košíku (`@Order` 4)

1. `highStockProduct(SHOP_A)`.
2. Otevře `/shop/shop-1`, `addProductByName(name, 3)` — tři kliknutí na „Přidat„ (nebo nastavení qty = 3).
3. Otevře `/cart`.
4. Assert: `cartPage.getCartItemCount() == 1` — sloučení do jednoho řádku s qty 3, ne tři samostatné řádky.

Test 5 — objednávka pod 50 Kč → „Minimální hodnota„ (`@Order` 5)

1. `cheapestAvailableProduct(SHOP_A)` — nejlevnější produkt.
2. Precondition assert: `price < 50`.
3. Otevře `/shop/shop-1`, `addProductByName(name, 1)`.
4. `fillAndSubmitCheckout()`: GET `/order/checkout`, vyplní Jana Horáková, jana.horakova@example.com, telefon, submit.
5. Čeká na redirect na `/cart`.
6. Assert: `.alert-danger` obsahuje `"Minimální hodnota"`.

Test 6 — dokončení objednávky jako host → potvrzení (`@Order` 6)

1. `productAndQtyForMinTotal(SHOP_A, 50)` — vypočítá `qty = ceil(50 / price)` a zvolí produkt, který má dost kusů na skladě (preferuje nejmenší qty).
2. Otevře `/shop/shop-1`, `addProductByName(name, qty)`.
3. `fillAndSubmitCheckout()`.
4. Čeká na URL `/order/confirmation`.
5. Assert: `confirmationPage.isOnConfirmationPage() == true`.

3.4. `AuthClientOrderTests`

3.4.1. Co testuje

Funkcionality dostupné pouze **přihlášenému uživateli** (jan).

1. **Odeslání objednávky** — end-to-end od přidání do košíku po potvrzení.
2. **Předvyplnění kontaktních údajů z profilu** — po kliknutí na tlačítko s třídou `btn-outline-ice` se musí pole `firstName`, `lastName`, `email` vyplnit z `User` účtu.
3. **Zobrazení vlastních objednávek** na `/order/moje` — po vytvoření musí být objednávka v seznamu.
4. **Zrušení objednávky ve stavu NEW** — kliknutí na cancel tlačítko → flash „zrušena„.

3.4.2. Jak je napsán

`@BeforeEach` volá `loginAsUser()`. Podobně jako `CreateOrderProcessTests` používá `ProductService` k výběru vhodného produktu přes `productAndQtyForMinTotal(SHOP_A, 50)`. Zrušení objednávky přes UI obchází `window.confirm` dialog JavaScriptem (`window.confirm = () => true`), protože Selenium by jinak na nativní confirm čekal.

3.4.3. Průběh jednotlivých testů

Test 1 — odeslání objednávky jako přihlášený → potvrzení (`@Order` 1)

1. `@BeforeEach`: `loginAsUser()`.
2. `addSufficientProductToCart()`: `productAndQtyForMinTotal(SHOP_A, 50)` → otevře `/shop/shop-1` → `addProductByName(name, qty)`.

3. `fillAndSubmitCheckout()`: GET `/order/checkout`, vyplní Jan Novák, jan@example.cz, telefon, submit.
4. Čeká na URL `/order/confirmation`.
5. Assert: `isOnConfirmationPage() == true`.

Test 2 — předvyplnění kontaktních údajů z účtu (`@Order` 2)

1. `addSufficientProductToCart()` (aby checkout šel vůbec zobrazit).
2. GET `/order/checkout`, čeká na přítomnost `#firstName`.
3. Klikne `button.btn-outline-ice` — tlačítko „Předvyplnit z účtu„, které přes JS vezme hodnoty z aktuálního `User` a zapíše do polí.
4. Assert: `#firstName.value == "Jan"`, `#lastName.value == "Novák"`, `#email.value == "jan@example.cz"`.

Test 3 — objednávka je viditelná v „Moje objednávky„ (`@Order` 3)

1. `addSufficientProductToCart()` → `fillAndSubmitCheckout()` → čeká na `/order/confirmation`.
2. `myOrdersPage.open(baseUrl())` — GET `/order/moje`.
3. Assert: `hasOrders() == true` — tabulka obsahuje alespoň jednu objednávku přihlášeného uživatele.

Test 4 — zrušení objednávky ve stavu Nová (`@Order` 4)

1. `addSufficientProductToCart()` → `fillAndSubmitCheckout()` → čeká na `/order/confirmation`.
2. `myOrdersPage.open(baseUrl())`.
3. Assert: `hasCancelButton() == true` — nově vytvořená objednávka (stav `NEW`) má cancel tlačítko.
4. `cancelFirstOrder()`:
   - Přes JS přepíše `window.confirm = () => true`, aby Selenium neblokoval na nativním confirm dialogu.
   - Klikne na tlačítko Cancel první objednávky.
5. Čeká na URL `/order/moje`.
6. Assert: `isSuccessDisplayed() == true` a `getSuccessText()` obsahuje `"zrušena"`.

3.5. `AdminPageTests`

3.5.1. Co testuje

Administrátorskou sekci dostupnou pouze s rolí `ROLE_ADMIN`.

1. **Změna stavu objednávky** NEW → CONFIRMED — nejdřív se programaticky vytvoří objednávka přes `OrderService` (bez UI), potom admin v `/admin/orders` změní `<select>` a odešle formulář.
2. **Vytvoření nové zmrzlinárny** — `/admin/shops/new`, vyplnit, submit.
3. **Přidání nového produktu** — `/admin/products/new`, přiřadit k zmrzlinárně, submit.
4. **Naskladnění produktu** — editace existujícího produktu, navýšení `stockQuantity` o 50 a ověření přes `productService.findById(...)`, že se změna uložila.

3.5.2. Jak je napsán

`@BeforeEach` volá `loginAsAdmin()`. Pro test změny stavu objednávky se objednávka vytváří **obejitím UI** — přímo přes `orderService.createOrder(...)` s naplněným `CartItem` a `CustomerDetails`. To je záměr: test se soustředí na admin flow, ne na znovuvytváření objednávky přes zákaznický flow (ten je pokryt jinde).

Tento přístup kontrastuje s `CreateOrderProcessTests`: tam je objednávka **předmětem** testu (prochází přes UI), tady je **předpokladem** (vytváří se programaticky, aby test měl na čem pracovat).

3.5.3. Průběh jednotlivých testů

Test 1 — změna stavu objednávky NEW → CONFIRMED (`@Order` 1)

1. `@BeforeEach`: `loginAsAdmin()` (admin/admin123).

2. `createTestOrder()` — **bez UI**: najde dostupný produkt v SHOP_A, spočítá `qty = ceil(50 / price)`, vyrobí `CartItem` a `CustomerDetails`, zavolá `orderService.createOrder(...)`. Vrátí `orderId`.
3. GET `/admin/orders`.
4. Najde formulář `form[action*='/admin/orders/status/{orderId}']` (odpovídá konkrétní objednávce).
5. V `<select name="status">` vybere hodnotu `"CONFIRMED"`.
6. Přes `JavascriptExecutor` zavolá `form.requestSubmit()` (odešle formulář programově, aby Selenium nemusel řešit který submit button).
7. Čeká na `.alert-success`.
8. Assert: text alertu obsahuje `"Status objednávky byl aktualizován."`.

Test 2 — vytvoření nové zmrzlinárny (`@Order` 2)

1. GET `/admin/shops/new`, čeká na `#name`.
2. `sendKeys` do `#name` → „Testovací Zmrzlinárna„.
3. `sendKeys` do `#address` → „Testovací ulice 1″.
4. `sendKeys` do `#city` → „Praha„.
5. Klik na `form[action*='/admin/shops/save'] button[type='submit']`.
6. Čeká na `.alert-success`.
7. Assert: text obsahuje `"Zmrzlinárna byla uložena."`.

Test 3 — přidání nového produktu (`@Order` 3)

1. GET `/admin/products/new`, čeká na `#name`.
2. `new Select(#shopId).selectByValue("shop-1")` — přiřadí produkt ke SHOP_A.
3. `sendKeys`: `#name` → „Testovací Kopečík„, `#flavor` → „Jahoda“, `#price` → „35″, `#stockQuantity` → „20″.
4. Klik na submit tlačítko formuláře `/admin/products/save`.
5. Čeká na `.alert-success`.
6. Assert: text obsahuje `"Produkt byl uložen."`.

Test 4 — naskladnění produktu (`@Order` 4)

1. Přes `productService.getAll()` najde první produkt v SHOP_A (mimo UI — pouze příprava dat).
2. Spočítá `newStock = currentStock + 50`.
3. GET `/admin/products/edit/{productId}`, čeká na `#stockQuantity`.
4. Přes `JavascriptExecutor` vymaže hodnotu pole (`arguments[0].value = ''`) — `sendKeys` by jen přidával ke stávající hodnotě.
5. `sendKeys(newStock)`.
6. Klik na submit.
7. Čeká na `.alert-success`, assert: text obsahuje `"Produkt byl uložen."`.
8. **Dvojitá verifikace**: `productService.findById(productId)` → `stockQuantity`.
9. Assert: hodnota v úložišti `== newStock` (ověření, že se změna skutečně propsala do `products.json`).

4. Souhrn — mapování testů na techniky

Třída	Technika (dle přednášek)	Úroveň testu
`IceCreamConfigurationPairwiseTests`	Pairwise (all-pairs) — přednáška 2	E2E přes Selenium
`CreateOrderProcessTests`	CPT / PCT TDL=2 — přednáška 3	E2E přes Selenium
`NoAccountClientOrderTests`	Funkční testy + CSV--driven (kvíz) — přednáška 5	E2E přes Selenium
`AuthClientOrderTests`	Funkční testy — happy path + zrušení objednávky	E2E přes Selenium
`AdminPageTests`	Funkční testy admin CRUD	E2E přes Selenium

4.1. Sdílené vlastnosti všech tříd

- Rozšiřují `SeleniumBaseTest` — získávají headless ChromeDriver a port aplikace.
- Používají `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` s `@Order(...)` — testy uvnitř třídy jsou na sobě záměrně závislé, protože sdílí stav v `./data/*.json`.
- Asertace používá knihovnu AssertJ (`assertThat(...).as(...).isTrue()`) pro čitelné chybové hlášky.
- `WebDriverWait` s 10s timeoutem pro čekání na async DOM změny.